Lab 4: Federated Identity, Observing and Breaking OIDC

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

Lab 3 answered ”who are you?” for a single machine. Modern systems delegate that question to an **identity provider**: you prove who you are once, to one trusted party, and other applications accept that party's word. This lab is about **OpenID Connect (OIDC)**, the identity layer over OAuth 2.0 that underpins most ”log in with…” buttons you meet. You will run a complete OIDC authorization-code flow against a **local identity provider**, capture and decode the tokens, understand what each token actually proves and to whom, and then examine **two classic misconfigurations** that turn a working login into a security hole. The lab stack runs inside the VM, and your own browser reaches it through SSH; no external service and no real account are involved.

A note on scope (the crypto boundary). You need to know what a *signature* proves and what a token's *issuer* and *audience* are. The half-lecture at the head of today's session covers exactly that. You do **not** need the mathematics of signatures; that belongs to the Cryptography course.

Contents

- Learning Objectives
- Before the lab (at home, ~30-45 min)
- Part A: The actors and the flow
- Part B: Run the flow and capture the tokens
- Part C: What the tokens prove
- Part D: Misconfiguration 1, `redirect_uri` validation
- Part E: Misconfiguration 2, failing to validate the ID token
- If it does not work
- Reflection
- References
- Deliverables

Learning Objectives

By completing this lab, students will be able to:

1. Trace an OIDC authorization-code flow end to end, naming each actor and each redirect, and distinguishing what happens in the browser from what happens server to server.
2. Decode an ID token (a JWT) and explain the security purpose of its `iss`, `aud`, `exp`, `sub` and `nonce` claims.
3. Tell apart what the ID token and the access token are for, in OAuth 2.0 / OIDC.
4. Explain two common OIDC failures, weak `redirect_uri` validation and failure to validate the ID token, and state what a correct implementation must check.

What from this lab is assessed. Four things, all handed in with **Lab report 2** at the end of the block (Lab 5): your answers to the preparation questions **P1 to P3**, your annotated **flow sketch** from Exercise 1a, your **answer to 5c**, and your **Reflection**. The other exercises are not handed in. They are the practice that Lab 5 and its report build on, so do them properly and keep your notes: Lab 5's reflection asks you to compare this lab's identity provider with an SSH certificate authority.

Before the lab (at home, ~30-45 min)

Do all of this *before* the session, especially starting the stack, which **pulls container images** and needs network and time. Arrive with the identity provider already serving its discovery document. Boot the VM and take a snapshot `lab4-clean`.

The lab stack is distributed as `lab4-files.zip` alongside this handout. Before the session you only start it: `docker-compose.yml` brings up the identity provider **dex** (`:5556`) and a minimal demo client (`:3000`), and `dex-config.yaml` configures dex. The other files are introduced in the part of the lab that uses them.

The diagram shows where the main pieces live, not the messages between them: drawing those is Exercise 1a. dex and the client run in containers inside the VM; their files, `dex-config.yaml` and `client.py`, sit in the VM and are mounted into them. You run `verify.py` and `verify_naive.py` yourself in Part E, from a shell in the VM, and `flow.py` only if you need the headless alternative in Part B. One `ssh` session gives you both that shell and the two forwarded ports your browser uses.

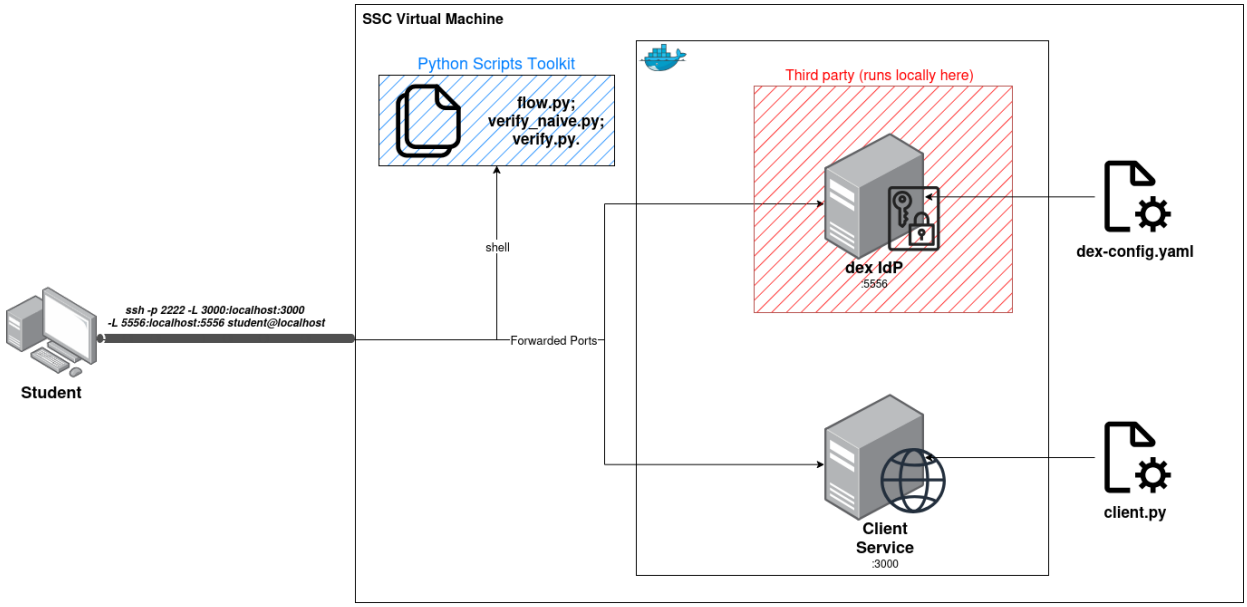


Figure 1: The lab stack: your computer, one ssh session with a shell and both ports forwarded, and inside the VM the scripts and the containers running dex and the demo client

The browser you will use is the one on your own computer: the VM has no desktop. So, **from a terminal on your own computer**, connect to the VM with the two lab ports forwarded, and keep that session open while you work:

```
ssh -p 2222 -L 3000:localhost:3000 -L 5556:localhost:5556 student@localhost
```

Copy the zip into the VM, from a second terminal on your own computer, in the folder where you downloaded it:

```
scp -P 2222 lab4-files.zip student@localhost:
```

Then, **inside the ssh session, in the VM**, start the stack:

```
mkdir -p ~/ssc-lab4 && cd ~/ssc-lab4
unzip ~/lab4-files.zip -d ~/ssc-lab4
docker compose up -d              # brings up dex (:5556) and the client (:3000)
docker compose ps                 # both 'dex' and 'client' should be running
```

Your own browser now reaches the client in the VM at `http://localhost:3000` and dex at `http://localhost:5556`. Check it before the session: `http://localhost:5556/dex/.well-known/openid-configuration` should open in your browser. If `ssh` says a port is already in use, stop whatever is using it on your computer: these addresses are registered in `dex-config.yaml`, so they have

to stay `localhost:3000` and `localhost:5556`. Do not run `docker compose` on your own computer: the stack belongs in the VM.

The values you will need. The provider image is pinned to `ghcr.io/dexidp/dex:v2.41.1`, which resolves on both architectures, and a cold `docker compose up` serves the discovery document. These are the values in the shipped `dex-config.yaml`, and the exercises below assume them: issuer `http://localhost:5556/dex`, client `democlient` and login `student@ssc.local` / `password`.

Read `dex-config.yaml` now. It is short enough to read in full, and you should: it declares the issuer, the static client (`democlient`, its secret and its **registered redirect URI**), and the static user account you will log in as. Everything this lab does to the provider, you can see in that one file.

The identity provider serves its OIDC **discovery document** at a well-known URL. Confirm it is up:

```
curl -s http://localhost:5556/dex/.well-known/openid-configuration \
  | jq '{issuer, authorization_endpoint, token_endpoint, jwks_uri}'
```

Preparation questions (submit with Lab report 2)

P2 is asked at the start of Part C, where you look at what the tokens prove, and is submitted with these.

- **P1.** Paste the discovery document's `issuer`, `authorization_endpoint` and `token_endpoint`, which is your proof that the identity provider is running before the session.
- **P3.** From `dex-config.yaml`, quote the **exact** `redirectURIs` entry registered for `democlient`. You will try to break it in Part D.

Part A: The actors and the flow

OIDC's authorization-code flow involves four parties: the **user** (in a browser), the **client** (the demo app, called the *Relying Party*), the **identity provider** (the *OpenID Provider*: authorization + token endpoints), and the protected **resource** the client wants to serve.

Exercise 1. Read the discovery document you fetched with `curl` in the pre-lab (the `.well-known/openid-configuration` URL, not `dex-config.yaml`). Identify the `authorization_endpoint` (where the browser is sent to log in), the `token_endpoint` (where the client swaps a code for tokens), and the `jwks_uri` (where the client fetches the provider's public keys to verify signatures).

1a. Sketch the flow as a sequence: user's browser → client → authorization endpoint → (login) → redirect back with a `code` → client → token endpoint → tokens. **After Exercise 2**, mark on your sketch which of these steps you saw in the browser's Network tab and which you did not, and explain why it is significant that the code-for-token exchange happens on the back channel.

Part B: Run the flow and capture the tokens

How the client works, and the headless alternative. The `client` service is `client.py`, a small stdlib relying party. `/login` sends your browser to dex with PKCE + `state` + `nonce`; dex redirects back to `/callback`, which does the code-for-token exchange **server-side** (the back channel) and shows the tokens at `/debug`. If you cannot use your own browser through the forwarded ports, `python3 flow.py` drives the identical flow headlessly and prints `ID_TOKEN=/ACCESS_TOKEN=`; note in your report if you used it. You then have no Network tab, and `flow.py` does not go through the browser at all, so in 1a mark which messages *would* go through the browser in the browser-and-client flow and which would go server to server, and say that this is reasoning rather than observation. Both need `docker compose up -d` running first (the client reaches dex on the VM's host network, so `localhost:5556` reaches the same `dex` from the client's back channel and, through the forwarded port, from your browser).

Exercise 2. In your own browser, with the `ssh` session from the pre-lab open, open the demo client (`http://localhost:3000`) and click ”Log in”. You are redirected to the identity provider; authenticate as the account declared in `dex-config.yaml`; you are redirected back to the client, now logged in.

Observe the redirects. Using the browser developer tools (Network tab, **preserve log**, without which the redirect chain is lost), find:

What `flow.py` does not give you. The observations below (the Network tab, the front-channel `code`, the *absent* back-channel token POST, the `/debug` page) come from the browser and the `client` at `:3000`.

2a. The request to the `authorization_endpoint`. Record its query parameters: `client_id`, `redirect_uri`, `response_type`, `scope`, `state`, `nonce`, and (the demo client uses PKCE) `code_challenge` and `code_challenge_method` (expect `S256`).

2b. The redirect back to the client's `redirect_uri`, carrying the `code`. Note that you can see the code in the browser, because the front channel is the browser. Then observe that the client's POST to the `token_endpoint` is **not** in the Network tab at all: it happened server-side. You explain why this matters in 1a. Retrieve the resulting tokens from the client's debug page at `http://localhost:3000/debug`.

2c. **Audit the demo client.** Read `client.py`, then:

1. List the checks a client must perform on an ID token that this one omits.
2. Say what an attacker who could mint or replay an ID token would get from this client, and why ”we use OIDC” is not a statement about security.

Part C: What the tokens prove

The token response contains (at least) an **ID token** and an **access token**, both JWTs. A JWT is three base64url segments, header, payload and signature, separated by dots.

- **P2** (submitted with P1 and P3). In one sentence, what does an **ID token** prove that an **access token** does not?

Exercise 3. Decode the ID token's header and payload (do not verify the signature by hand; just decode):

```
# Paste the ID token into $ID. JWTs use base64URL (with -/_ and no '=' padding),
# which is NOT plain base64, so `base64 -d` fails on it. Decode with a helper:
decode() { python3 -c "import base64,sys; s=sys.argv[1];
 ↪  print(base64.urlsafe_b64decode(s+'='*(-len(s)%4)).decode())" "$1"; }
decode "$(echo "$ID" | cut -d. -f1)" | jq .    # header: alg, kid
decode "$(echo "$ID" | cut -d. -f2)" | jq .    # payload: the claims
```

3a. From the payload, record and explain the security role of: `iss` (issuer), `aud` (audience), `exp` (expiry), `sub` (subject), and `nonce`. For each, state the attack that the client *fails to stop* if it ignores that claim. (Answer each in the form: if the client ignores <`claim`>, an attacker can ...)

3b. The header names an algorithm (`alg`) and a key id (`kid`). How does the client know which public key to use to verify the signature, and where does it get that key (tie back to `jwks_uri` in Part A)? Fetch the JWKS and confirm the `kid` matches:

```
curl -s http://localhost:5556/dex/keys | jq '.keys[] | {kid, alg, kty}'
```

3c. **Authentication vs authorisation.** Building on your answer to P2, explain in one or two sentences why treating an *access token* as proof of identity (a very common mistake) is wrong.

Part D: Misconfiguration 1, `redirect_uri` validation

The provider must only redirect the authorization `code` back to a **pre-registered** `redirect_uri`. Loose validation lets an attacker have the code, and thus potentially the tokens, delivered to a URL they control.

Exercise 4. Attempt the flow while tampering with the `redirect_uri`. The URL below is a minimal authorization request for `democlient` (no PKCE, fixed `state` and `nonce`) whose `redirect_uri` is the tampered one. It is too long for one line of this page, so let the VM's shell assemble it, then copy the line it prints into your browser:

```
AUTH=http://localhost:5556/dex/auth
Q='client_id=democlient&response_type=code&scope=openid+profile+email'
Q="$Q&state=x&nonce=y"
Q="$Q&redirect_uri=http://localhost:3000.attacker.example/callback"
echo "$AUTH?$Q"
```

⚠ **Read that `redirect_uri` carefully, because it is chosen and not arbitrary.** `http://localhost:3000.attacker.example/callback` is built to defeat a **prefix** rule: anything checking "does it start with `http://localhost:3000`" says yes. ⚠ **But it is not a working exfiltration target**: after `localhost:`, URI syntax expects a *port*, and `3000.attacker.example` is not one, so a strict parser gets host `localhost` and fails on the port. Keep apart what a matching rule decides and what a URI parser rejects: this string demonstrates *which matching rules it would defeat*. A working attack on a prefix rule needs a URI that still starts with the prefix and yet delivers the code to the attacker. With a prefix that carries a port, like this one, a remote attacker has no straightforward URI of that kind, because a host name cannot follow the port: one more reason 4b asks you to reason about the attack rather than run it.

4a. Watch what `dex` does with this.

1. Record **exactly** what the browser shows, including the exact text of any error. Which step that you saw in Exercise 2 is absent here?
2. In the Network tab (with **preserve log**), record the first two responses to the authorization request: the status of each, and its `Location` header or what its body says. Does the first one mean that `dex` accepted your `redirect_uri`? Explain.
3. Compare the tampered value with the registered one you quoted in P3, and say precisely which comparison failed.

4b. Explain the full attack **when validation is loose**: an attacker persuades a logged-in victim to open an authorization URL whose `redirect_uri` points at attacker-controlled infrastructure that a sloppy matching rule still accepts (a prefix match, a wildcard subdomain, an unanchored substring). Trace where the victim's `code` ends up and what the attacker does with it next.

4c. Consider two cases.

- **(i)** The legitimate client started the flow and generated the `code_verifier`; the victim's `code` then leaks to the attacker. What else must the attacker present at the token endpoint to redeem it, and which of those never travels through the browser?
- **(ii)** The attacker builds the authorization URL themselves, with a `code_challenge` of their own choosing, for a **public** client (one with no secret), and the victim logs in through it. What does PKCE protect in this case, and what is left standing between the attacker and the victim's tokens?

4d. 4b named three sloppy matching rules, and the URI in Exercise 4 is built to defeat the first, a prefix match. Suppose the registered URI is `https://app.example.com/callback`.

1. For a **wildcard subdomain** rule (`https://*.example.com/callback`), write a `redirect_uri` that the rule accepts and that sends the code somewhere the attacker controls. Say what the attacker needs for it to work.
2. Do the same for an **unanchored substring** rule (any URI containing `app.example.com`).
3. Name the rule a provider should implement, and say why it closes all three.

On seeing a defence rather than a breach. This exercise deliberately shows you a *correct* implementation refusing an attack. That is the honest outcome, and it is worth knowing what a provider doing its job looks like. The vulnerable case is an *implementation or configuration* choice, not a protocol flaw, and you will meet it as a demonstration rather than reproduce it, by seeing the strict case refuse and then reasoning about what a loose one would allow.

There is no loose case to break here. Some implementations relax the rule for public `localhost` clients

and accept any port. `dex v2.41.1` does not: registered either as `http://localhost:3000/callback` or portless, it refuses any other port on `localhost`, and `127.0.0.1` and `[::1]` as well. Part D shows you a **correct** provider refusing the attack, and 4b asks you to reason about what a loose implementation would have allowed.

Part E: Misconfiguration 2, failing to validate the ID token

An ID token is only meaningful if the client **validates** it. A client that decodes the payload without verifying trusts anything.

⚠ **The demo client is one of those.** As you found in 2c, `client.py` decodes the ID token's payload and shows you the claims without validating the token. In Part B it *sent* PKCE, `state` and `nonce`: sending a parameter is not checking it, and its presence in the request proves nothing about the client. This part gives you the verifier that does the job, `verify.py`, and the naive one beside it, `verify_naive.py`, both in `lab4-files.zip`.

Exercise 5. Using the decoded token from Part C, construct a **tampered** ID token: change the `sub` (or an email or username claim) in the payload to impersonate another user, re-encode the payload as base64url, and leave the signature untouched, which makes it invalid.

Concretely, with the ID token in `$ID` (from Part C, or from `flow.py`):

```
# build a tampered copy: change email/sub in the payload, keep the signature
TAMP=$(python3 - "$ID" <<'EOF'
import sys, base64, json
h, p, s = sys.argv[1].split(".")
d = lambda x: base64.urlsafe_b64decode(x + "=" * (-len(x) % 4))
e = lambda b: base64.urlsafe_b64encode(b).rstrip(b"=").decode()
c = json.loads(d(p)); c["email"] = "attacker@evil.example"; c["sub"] = "attacker"
print(".".join([h, e(json.dumps(c, separators=(",", ":")).encode()), s]))
EOF
)
python3 verify.py "$TAMP"
python3 verify_naive.py "$TAMP"
```

5a. Present the tampered token to `verify.py` (provided), which validates it against the `jwks_uri`. Record and explain the result.

5b. Now run the same tampered token through `verify_naive.py` (provided), which base64-decodes the payload and trusts what it finds. Record the result, and explain precisely which check the naive verifier skipped, and why ”the `alg: none` attack” and ”just decode the JWT” are two faces of the same failure.

5c. State the minimum set of validations a correct OIDC client must perform on an ID token before trusting it.

Restore `lab4-clean` (or run `docker compose down`) when finished.

If it does not work

Two of these look like a broken lab and are not: the provider is behaving correctly and that is what you are meant to observe.

What you see	What to do
The browser shows an error as soon as you open the tampered URL	That is not the lab breaking. Record the error and the responses in the Network tab, as 4a asks.

What you see	What to do
You cannot find a loopback case where the provider relaxes the match	There is not one. `dex v2.41.1` does not relax its rule, even for public loopback clients. Part D shows a correct provider refusing; 4b is where you reason about what a loose provider would have allowed.
Login fails with `Invalid Email Address and password`	The login is the email address, `student@ssc.local`, not a username. Use the credentials in the shipped `dex-config.yaml`.

If none of these matches, read the error message in full first: it usually names the layer that refused, and that is most of the answer. Then ask your lab teacher, in the session or by email.

Reflection

Your reflection goes into Lab report 2, as an appendix, beside the reflections from the other labs in this block. They are assessed with the report: every one has to be there, each answering its questions with a position you argue from what you did in the lab.

In a short piece (200 to 300 words):

1. OIDC moves the authentication decision *off* your machine and onto a trusted third party. What does that centralisation buy, and what does it concentrate: what is now the consequence of the identity provider being compromised? (Lab 5 makes the same move for *host* access with an SSH CA. Keep your answer; you will compare them.)
2. Suppose a developer fixes `client.py` by passing every ID token it receives through `verify.py`, so that every check in 5c now holds, and declares the client secure. Its `state` store is unchanged: it is **global**, refusing a `state` it has never seen but not tying the response to the browser that started the flow. Using what you read in `client.py`, say what an attacker can still do to a user of this client, and why a token that passes every check is not yet a secure login. What does that tell you about where ”validating the token” stops being enough?

References

- OpenID Connect Core 1.0, https://openid.net/specs/openid-connect-core-1_0.html
- OAuth 2.0 (RFC 6749) and OAuth 2.0 Security Best Current Practice (RFC 9700).
- PKCE (RFC 7636).
- JWT (RFC 7519); JWS (RFC 7515), including the `alg: none` pitfall.
- `dex` documentation, https://dexidp.io/docs/: connectors, static clients, and the redirect-URI matching rules.
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022: the web-authentication material.

Deliverables

There is **no separate graded lab report for this lab**. Block 2's assessed deliverable is **Lab report 2**, produced in Lab 5. Your **preparation questions P1-P3 are submitted with it** (required), together with your annotated flow sketch from Exercise 1a, your answer to 5c and your **Reflection**. **The sketch goes inside the report, as an image.** A photograph of a hand drawing is fine if it is legible at A4; a file only a particular tool can open is not a submission. Keep the rest of your notes: Reflection question 1 is the input to Lab 5's own reflection, which asks you to compare this centralisation with an SSH certificate authority.